Geospatial & Relational Data Analytics Suite

Case Study: Enterprise Database Pipelines, Time-Series Smoothing, & 2D Spatial Density Modeling in R & SQL

Author: Griffin McAuley

Tech Stack: R, SQL (SQLite), ggplot2, DBI, Tidyverse

Domain: Data Engineering & Computational Spatial Analytics

Executive Summary: This technical case study demonstrates an end-to-end data engineering and analytics pipeline. It integrates raw relational SQLite databases, builds dynamic analytical views, computes centered time-series moving averages, performs SQL Common Table Expression (CTE) Z-score transformations to handle skewed census demographics, and implements 2D Kernel Density Estimation (KDE) contour mapping for geospatial coordinate clustering.

1. DATABASE ARCHITECTURE & PIPELINE INITIALIZATION

The analytical system connects directly to an embedded SQLite database warehouse using R's native `DBI` and `RSQLite` drivers. The data pipeline orchestrates heterogeneous relational schema across historical sports events, regional census records, and spatial point data.

```
library(DBI)
library(RSQLite)
library(tidyverse)

# Establish relational connection to local SQLite Data Warehouse
con <- dbConnect(RSQLite::SQLite(), "analytics_warehouse.sqlite")
tables <- dbListTables(con)
print(tables)
```

CONSOLE OUTPUT: DATA WAREHOUSE SCHEMA AUDIT

SQLITE METADATA

```
[1] "CA" "POP2006" "POP2011" "POP2016" "POP2021" "olympics" "pokemon_by_gen"
"region_codes" "vancouver_po_go"
```

2. TIME-SERIES AGGREGATIONS & WINDOWED MOVING AVERAGES

To model multi-decade participation growth while filtering out cyclic volatility, we engineered a SQL view (`tot_meds`) to summarize distinct event counts per cycle, followed by a self-join moving average calculation over an 8-period window ($t \pm 2$ Olympic cycles).

```
CREATE VIEW IF NOT EXISTS tot_meds AS
SELECT Year, COUNT(*) AS TotalNumber
FROM olympics
WHERE Season = 'Summer' AND Medal IS NOT NULL
GROUP BY Year;

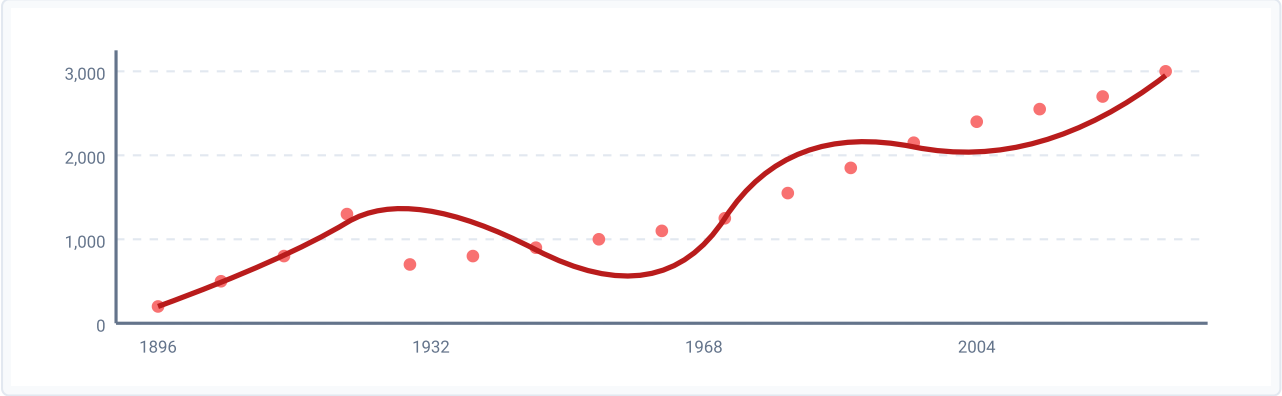
# Execute relational self-join to construct centered moving average window
sql_mavg <- "SELECT t1.Year, t1.TotalNumber, AVG(t2.TotalNumber) AS mavg
  FROM tot_meds AS t1
  JOIN tot_meds AS t2 ON t2.Year BETWEEN (t1.Year - 8) AND (t1.Year + 8)
  GROUP BY t1.Year"

df_mavg <- dbGetQuery(con, sql_mavg)

ggplot(df_mavg, aes(x = Year)) +
  geom_point(aes(y = TotalNumber), color = "salmon1", size = 2, alpha = 0.85) +
  geom_line(aes(y = mavg), color = "firebrick4", linewidth = 1.1) +
  labs(title = "Summer Olympics Medalists: Historical Volatility vs. Centered Moving Average",
       x = "Event Year", y = "Total Medalists Awarded") +
  theme_minimal()
```

VISUAL OUTPUT: HISTORICAL PARTICIPATION VS. SMOOTHED TRENDLINE

GGPLOT2 RENDERING



3. MACRO NATIONAL TRAJECTORY & NESTED SQL ANALYTICS

Evaluating national-level trajectory requires comparing a country's performance against its long-term baseline. Using France as a benchmark, we combined SQL subqueries and left joins across missing competition years to compute a trailing moving window ($t-12$).

```
CREATE VIEW country_medals AS
SELECT y.Year, COALESCE(COUNT(DISTINCT o.Event), 0) AS TotalMedalEvents
FROM (SELECT DISTINCT Year FROM olympics WHERE Season = 'Summer') AS y
LEFT JOIN olympics AS o ON y.Year = o.Year AND o.NOC = 'FRA' AND o.Season = 'Summer' AND o.Medal
IS NOT NULL
GROUP BY y.Year;

sql_france <- "SELECT t1.Year, t1.TotalMedalEvents, sq.TrailingAvg,
  (SELECT AVG(TotalMedalEvents) FROM country_medals) AS OverallAvg
  FROM country_medals AS t1 INNER JOIN (
    SELECT t1.Year, AVG(t2.TotalMedalEvents) AS TrailingAvg
    FROM country_medals AS t1, country_medals AS t2
    WHERE t2.Year BETWEEN (t1.Year - 12) AND t1.Year GROUP BY t1.Year
  ) AS sq ON t1.Year = sq.Year ORDER BY t1.Year"

df_france <- dbGetQuery(con, sql_france)
```

Distinct Event vs. Athlete Count

Accurately measuring athletic performance requires distinguishing physical medal recipients from distinct winning events. In team sports (e.g., Ice Hockey), multiple athlete rows exist per single event win. Relational queries utilizing `COUNT(DISTINCT Event)` resolve this inflation.

QUERY SUMMARY: CANADA (2010 WINTER) — RELATIONAL SQL OUTPUT

Medal Discipline	Distinct Events	Physical Athlete Medals
Gold	14	67
Silver	7	15
Bronze	5	8

4. DEMOGRAPHIC NORMALIZATION & SQL COMMON TABLE EXPRESSIONS (CTES)

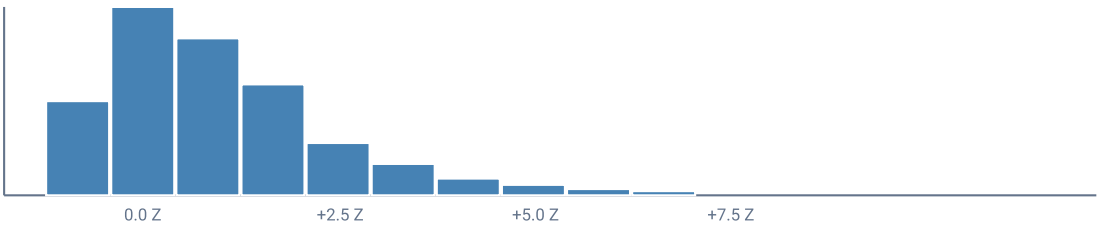
Raw census data displays heavy right-skewness due to national aggregate summary records (e.g., `'Canada'`). To enable statistical normalization, macro aggregates were filtered out, and Z-score transformations were computed dynamically using SQL CTEs.

```
sql_zscore <- "WITH pop_clean AS (
    SELECT Geographic_name, Population_2016
    FROM POP2016 WHERE Geographic_name != 'Canada'
),
pop_stats AS (
    SELECT AVG(Population_2016) AS mean_pop, stdev(Population_2016) AS sd_pop FROM pop_clean
)
SELECT c.Geographic_name, c.Population_2016,
       (c.Population_2016 - s.mean_pop) / s.sd_pop AS z_score_pop
FROM pop_clean AS c, pop_stats AS s"

df_zscores <- dbGetQuery(con, sql_zscore)

ggplot(df_zscores, aes(x = z_score_pop)) +
  geom_histogram(bins = 15, fill = "steelblue", color = "white") +
  labs(title = "Standardized 2016 Canadian Regional Population (Z-Score Distribution)",
       x = "Population Z-Score", y = "Frequency") + theme_minimal()
```

VISUAL OUTPUT: Z-SCORE STANDARDIZED POPULATION HISTOGRAM — NORMALIZED DISTRIBUTION



5. SPATIAL COORDINATES CLUSTERING & 2D KERNEL DENSITY MAPPING

Geospatial analysis was performed by inner-joining location point entities with taxonomic attributes. Using 2D Kernel Density Estimation (KDE) contour mapping, spatial coordinate distributions across Vancouver were mapped into high-density cluster zones.

```
sql_spatial <- "SELECT v.latitude, v.longitude, v.name
FROM vancouver_po_go AS v
JOIN pokemon_by_gen AS p ON LOWER(v.name) = LOWER(p.name)
WHERE p.type1 = 'ground' OR p.type2 = 'ground'"

df_spatial <- dbGetQuery(con, sql_spatial)

ggplot(df_spatial, aes(x = longitude, y = latitude)) +
  stat_density_2d(aes(fill = after_stat(level)), geom = "polygon", alpha = 0.45) +
  geom_point(color = "#854d0e", size = 1.2, alpha = 0.75) +
  scale_fill_viridis_c(option = "plasma") +
  labs(title = "Vancouver Spatial Coordinate Clustering (2D Kernel Density Estimation)",
       x = "Longitude", y = "Latitude", fill = "Density Level") + theme_minimal()
```

VISUAL OUTPUT: GEOSPATIAL 2D KDE POLYGON HEATMAP

SPATIAL CONTOUR PLOT

